

```
=====
MONGO ARCHIVER – Migration Plan (v6)
Automated backfill with resume-on-restart capability
=====
```

```
=====
TL;DR
=====
```

1. Build: `mvn clean package -DskipTests`
2. Deploy: `java -jar target/mongo-archiver-1.0.0.jar`
3. That's it. Backfill runs automatically. If it crashes, next restart resumes.

```
=====
STEP BY STEP
=====
```

STEP 1: Build the JAR

---

```
mvn clean package -DskipTests
```

```
Creates: target/mongo-archiver-1.0.0.jar
```

STEP 2: Deploy (first run)

---

```
MONGO_URI=mongodb://your-host:27017 \
MONGO_DB=yourdb \
java -jar target/mongo-archiver-1.0.0.jar
```

What happens on startup:

- a) Spring Boot initialises
- b) `ArchiveCronJob.init()` runs
  - └ `BackfillService.runBackfillIfNeeded()` is called
- c) For each job (orders, invoices...):
  - └ Query anchor collection for oldest record
  - └ Check if backfill is needed
  - └ If yes, calculate monthly slices
  - └ Run slices sequentially
    - | └ After each slice completes, progress is saved to `backfillState`
  - └ Mark backfill as COMPLETED when done

d) Scheduler becomes active

└ Daily 2AM runs begin (picks up only new eligible records)

Backfill runs in the foreground during startup. Don't kill it mid-way.

### STEP 3: Monitor progress

---

While backfill is running:

```
db.backfillState.find().pretty()
```

Example output during backfill:

```
{
  "jobName"           : "orders",
  "status"            : "IN_PROGRESS",
  "completedSlices"   : 7,
  "totalSlices"       : 36,
  "lastCompletedSliceEnd" : ISODate("2022-08-01T00:00:00Z"),
  "startedAt"         : ISODate("2025-07-01T02:00:00Z")
}
```

Check for archive failures:

```
db.archiveLog.find({ status: { $in: ["FAILED","PARTIAL_FAILURE"] } })
```

If slices are failing, check logs:

```
tail -f logs/archiver.log | grep -i "FAILED\\|ERROR"
```

### STEP 4: Restart safety

---

If the app crashes or is restarted mid-backfill:

- On next startup, BackfillService loads backfillState
- Finds lastCompletedSliceEnd (e.g., 2022-08-01)
- Resumes from the next slice (2022-08-01 → 2022-09-01)
- Does NOT start over from the beginning
- Progress is saved after each slice ← guaranteed resumability

You can safely restart the pod, redeploy, etc. Backfill always resumes.

### STEP 5: After backfill completes

---

backfillState status changes from IN\_PROGRESS → COMPLETED

Example final state:

```
{
  "jobName"      : "orders",
  "status"       : "COMPLETED",
  "completedSlices" : 36,
  "totalSlices"   : 36,
  "completedAt"   : ISODate("2025-07-01T05:42:00Z")
}
```

From this point:

- Subsequent restarts skip backfill entirely
- Only the daily scheduler runs
- Each day picks up records that became eligible that day
- Volume is small and predictable

=====

WHAT IF SOMETHING FAILS?

=====

SCENARIO: A backfill slice fails mid-way

---

Logs show:

```
[Archiver:orders] FAILED 'orderPayments' for entity 'ORD-7824'
```

Check archiveLog:

```
db.archiveLog.find({ "collections.orderPayments": "FAILED" })
```

Options:

- Let it continue (next startup resumes from that slice, maybe succeeds then)
- Investigate & fix the issue, restart app (resumes from same slice)
- Manually skip that slice:

```
db.backfillState.updateOne(
  { jobName: "orders" },
  { $set: { lastCompletedSliceEnd: ISODate("2022-08-01") } }
)
```

Then restart.

SCENARIO: Backfill is taking too long

---

Check batch-size in application.yml:

```
jobs:
```

```
- name: orders
  batch-size: 100    ← reduce this to 50 or 25 if DB is overloaded
```

Restart with smaller batch-size:

- Slices still run monthly, but each batch is smaller
- Slower but lighter on the DB

SCENARIO: I want to restart backfill from scratch

---

Delete the backfillState document:

```
db.backfillState.deleteOne({ jobName: "orders" })
```

Next startup will:

- See no state
- Query for oldest record again
- Recalculate slices and run from the beginning

SCENARIO: One job succeeded, another failed. Can I skip it?

---

Yes. backfillState is per-job. Other jobs continue independently.

If orders succeeded but invoices failed:

```
db.backfillState.find({ jobName: "invoices" })
→ still IN_PROGRESS, resume from there
db.backfillState.find({ jobName: "orders" })
→ COMPLETED, scheduler takes over
```

---

## MONITORING QUERIES

---

-- Overall backfill status --

```
db.backfillState.find({}, { jobName:1, status:1, completedSlices:1,
totalSlices:1 })
```

-- Estimated completion (rough) --

```
db.backfillState.find({}).forEach(doc => {
  let percent = (doc.completedSlices / doc.totalSlices * 100).toFixed(1);
  print(doc.jobName + ": " + percent + "% (" + doc.completedSlices + "/" +
doc.totalSlices + ")");
})
```

```

-- Any archive failures --
db.archiveLog.aggregate([
  { $match: { status: { $in: ["FAILED","PARTIAL_FAILURE"] } } },
  { $group: { _id: "$jobName", count: { $sum: 1 } } }
])

-- Which entity/collection combo failed --
db.archiveLog.find({ "collections.orderPayments": "FAILED" }, { entityId:1,
collections:1 })

-- Archive log growth (sanity check) --
db.archiveLog.countDocuments()

-- Backfill duration (how long did it take) --
db.backfillState.find({
  "completedAt": { $exists: true }
}, {
  jobName:1,
  startedAt:1,
  completedAt:1
}).forEach(doc => {
  let duration = (doc.completedAt - doc.startedAt) / 1000 / 60;
  print(doc.jobName + ": " + duration.toFixed(1) + " minutes");
})

```

---

#### CONFIGURATION NOTES

---

application.yml settings that matter for backfill:

1. archiver.schedule  
Controls when the daily scheduler runs.  
Default: "0 0 2 \* \* ?" (2 AM UTC)  
Change to "0 0 2 \* \* ?" with timezone: Asia/Kolkata for 2 AM IST.
2. archiver.timezone  
Timezone for the cron expression.  
Set to Asia/Kolkata for India.
3. jobs[].batch-size  
Number of entity IDs to process in one batch.  
Default: 100  
For heavy loads, reduce to 50 or 25.  
Affects both backfill and daily runs.

- 4. jobs[].anchor.archive-after-days
  - Retention window in days.
  - Default: 180 (6 months)
  - Records older than this are archived.

These are in the base config. No special flags needed for backfill.

```
=====
EXPECTED BEHAVIOR - Timeline
=====
```

DAY 1, 2AM - Deployment

- └ App starts
- └ BackfillService runs
  - └ Finds oldest record (Jan 2022)
  - └ Calculates 36 monthly slices
  - └ Slice 1: Jan 2022 → Feb 2022 
- └ Pod restarts (e.g., crash)
- └ App resumes from lastCompletedSliceEnd = 2022-02-01
  - └ Slice 2: Feb 2022 → Mar 2022 
  - └ Slice 3: Mar 2022 → Apr 2022 
  - └ ...

DAY 2, 2AM

- └ Slice N complete, backfillState.status = COMPLETED
- └ Scheduler takes over
  - └ Picks up records created on Jun 2025 (now - 180 days)

DAY 3, 2AM onwards

- └ Daily scheduler continues forever
  - (picks up records created exactly 180 days ago each day)

```
=====
ROLLBACK (if needed)
=====
```

If the archiver is not working as expected:

1. Stop the app
2. Delete all archive collections:
  - db.orders\_archive.deleteMany({})
  - db.orderItems\_archive.deleteMany({})
  - ...
3. Delete archiveLog:
  - db.archiveLog.deleteMany({})
4. Delete backfillState to start over:
  - db.backfillState.deleteMany({})

5. Restart:

```
java -jar mongo-archiver-1.0.0.jar
```

Source collections (orders, orderItems, etc.) remain unchanged.

```
=====
END OF MIGRATION PLAN
=====
```